

Lớp và đối tượng trong Java

(Dựa trên slide của PGS. TS. Nguyễn Việt Hà)

Nội dung

- Ngôn ngữ lập trình Java
- Định nghĩa lớp
- Thuộc tính
- Phương thức
- Kiểm soát truy cập
- Phương thức khởi tạo
- Thao tác với đối tượng

Tài liệu tham khảo

▣ *Giáo trình Lập trình HĐT, chương 3, 4*

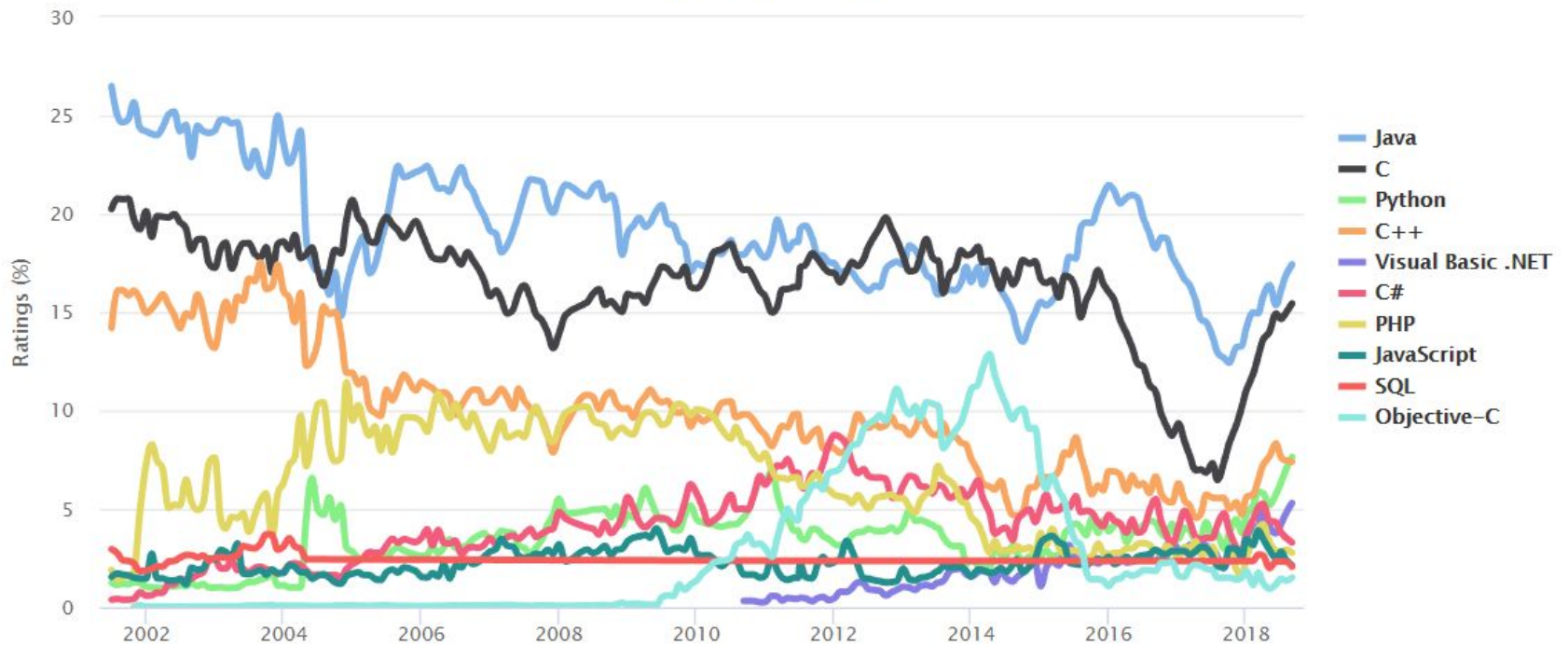
Java

- 1991: phát triển bởi Sun Microsystems như là một ngôn ngữ lập trình cho môi trường nhúng
 - Tên ban đầu là Oak
- Java 1.0.2, 1.1
 - “Write Once, Run Anywhere”
 - Chậm
 - Được sử dụng cho các ứng dụng Web (applets)
- Java 2 (versions 1.2 - 1.4)
 - Nhanh và mạnh hơn,
 - 3 nền tảng: J2ME, J2SE, J2EE
- Java 5, 6, 7, 8 (versions 1.5-1.6...)
 - Được cải tiến nhiều hơn nữa

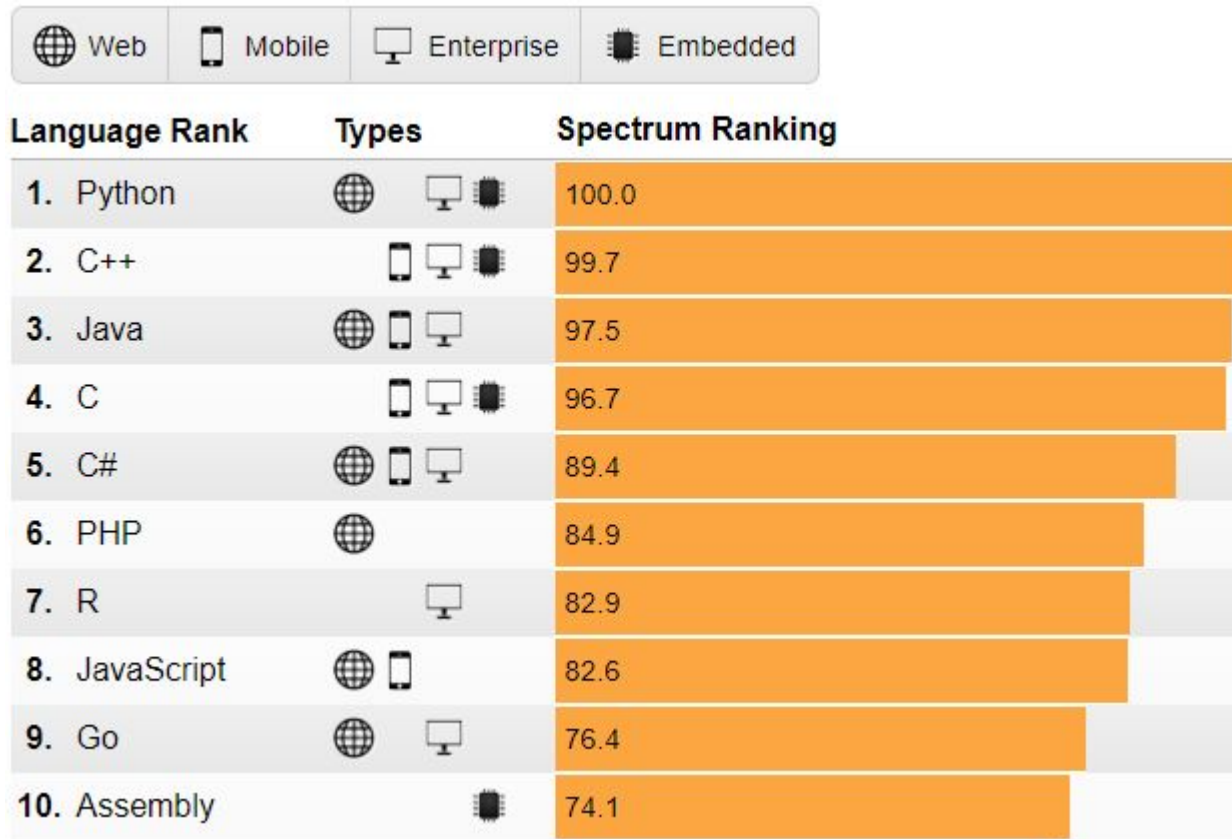
Thống kê về ngôn ngữ lập trình

TIOBE Programming Community Index

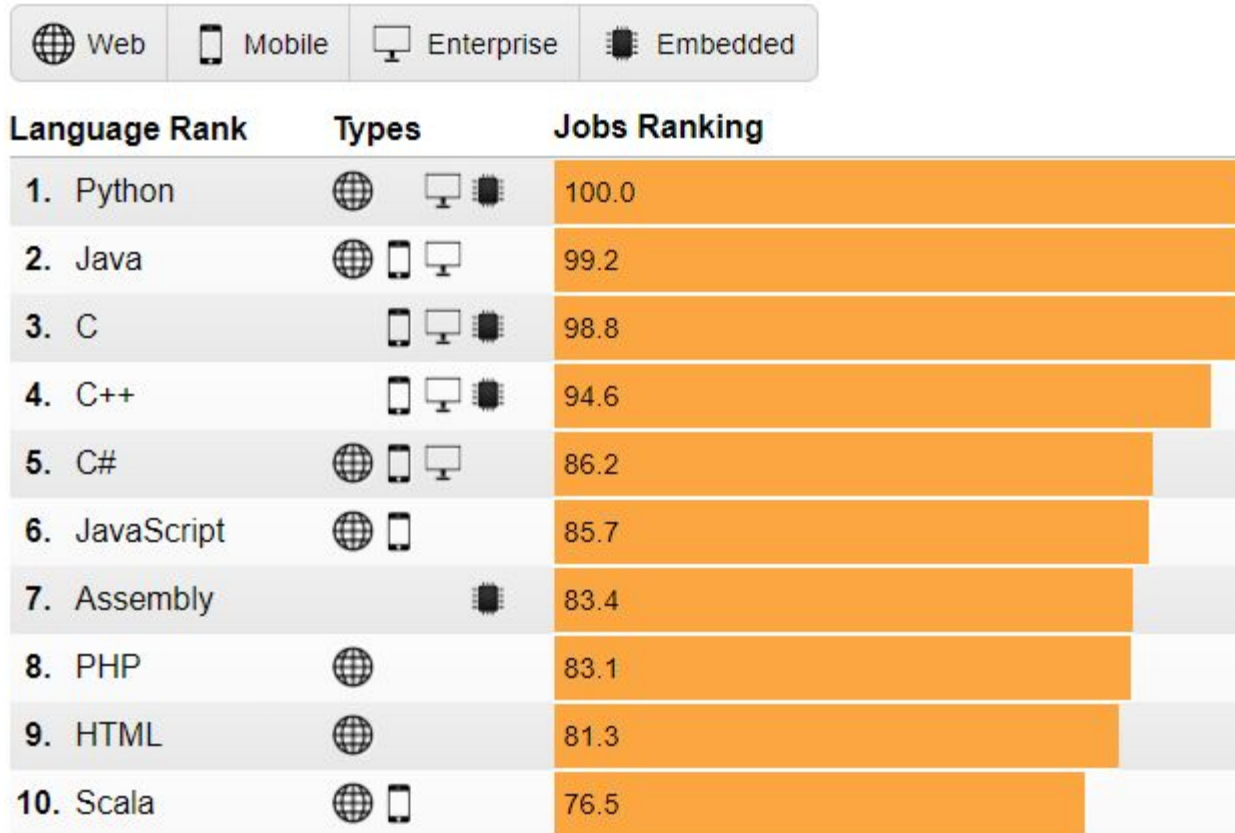
Source: www.tiobe.com



Thống kê của IEEE



Thống kê của IEEE (theo nhu cầu tuyển dụng)

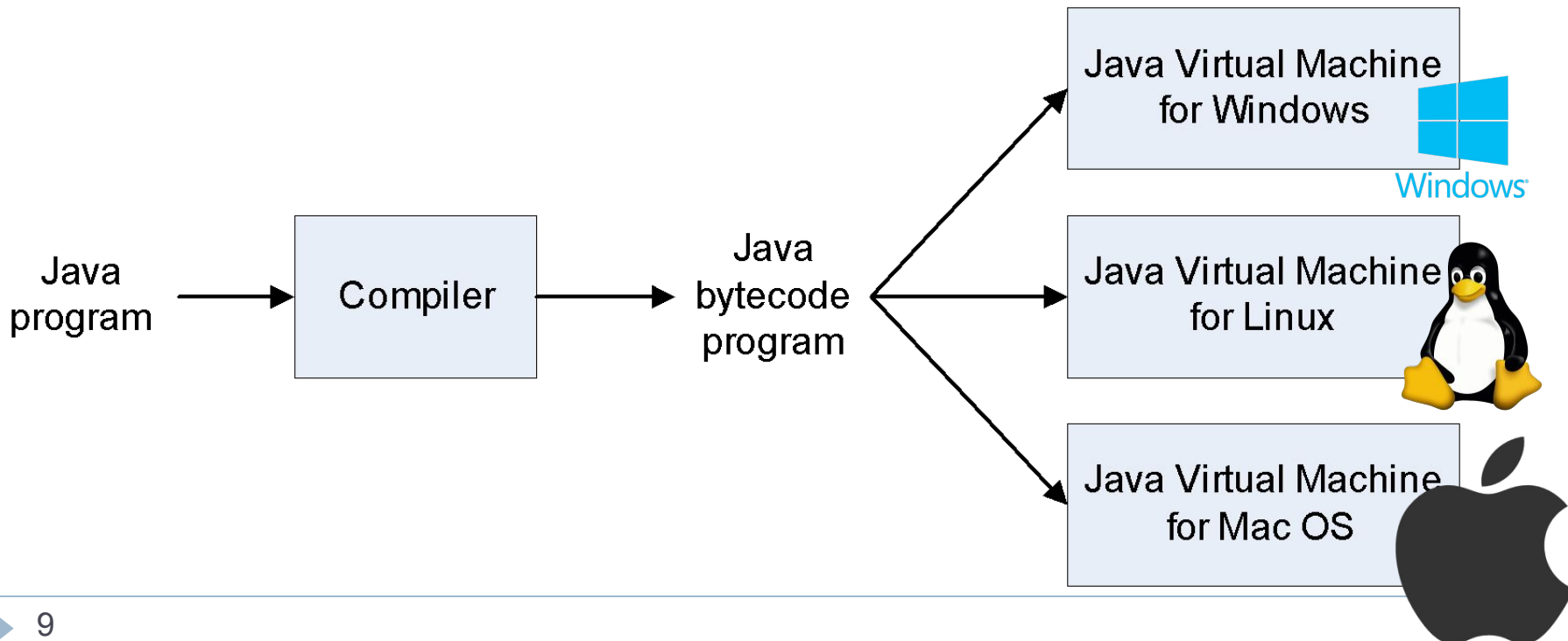


Cấu trúc của chương trình Java

- Tập hợp của các lớp đối tượng
- Thông thường mỗi lớp được mô tả trong 1 tệp mã nguồn (trùng tên với tên lớp)
 - Tăng tính độc lập
 - Dễ sửa đổi, tiết kiệm thời gian biên dịch

Biên dịch

- ❑ Mã nguồn Java được biên dịch thành mã bytecode
- ❑ Bytecode độc lập với nền tảng
- ❑ Bytecode được thực thi qua máy ảo Java (virtual machine)



Java Virtual Machine (JVM)

- JVM được hiện thực phụ thuộc vào các nền tảng (hardware, OS)
- Đảm bảo các chương trình Java (bytecode) có thể thực thi trên các nền tảng khác nhau (platform-independent)
- Đảm bảo an ninh
- Thường được hiện thực như là một phần mềm
 - JRE - Java Runtime Environment
- Java platform: JVM + APIs

Các loại ứng dụng Java

- Ứng dụng để bàn – *Java Standard Edition (Java SE)*
- Ứng dụng phân tán, ứng dụng chủ – *Java Enterprise Edition (Java EE)*
- Ứng dụng di động – *Java Micro Edition (Java ME)*
- Ứng dụng trên thẻ - Java Card

HelloWorld application

HelloWorld.java:

Lớp
↓
public class HelloWorld {

public static void main (String[] args) {

System.out.println("Hello, world");

}

}

trùng tên lớp

Tên lớp

Tên phương thức

*Phát biểu trong
phương thức*

*public, từ khóa quy
định mức truy xuất*

Biên dịch và thực thi

```
public class HelloWorld {  
    public static void main (String[] args)  
    {  
        System.out.println("Hello, world");  
    }  
}
```

- Compile HelloWorld.java

```
javac HelloWorld.java
```

- Run

```
java HelloWorld
```

compiler

HelloWorld.class

```
%> javac HelloWorld.java  
%> java HelloWorld  
Hello, world
```

Ứng dụng với nhiều hơn một lớp

2 lớp được hiện thực bởi 2 file

TestGreeting.java:

```
public class TestGreeting {  
    public static void main(String[] args) {  
        Greeting gr = new Greeting();  
        gr.greet();  
    }  
}
```

Greeting.java:

```
public class Greeting {  
    public void greet() {  
        System.out.print("Hi there!");  
    }  
}
```

Biên dịch và thực thi

□ Compile

```
javac TestGreeting.java
```

□ Greeting.java được dịch tự động

□ Run

```
java TestGreeting
```

```
%> javac TestGreeting.java
%> java TestGreeting
Hi there!
```

JDK – Java Development Kit

- Môi trường phát triển ứng dụng Java
- Các thành phần chính
 - *javac*: trình biên dịch, chuyển mã nguồn Java thành bytecode
 - *java*: trình thông dịch
 - *javadoc*: công cụ sinh tài liệu từ các chú thích trong mã nguồn
 - *jdb*: trình gỡ lỗi

Định nghĩa lớp

Cú pháp

```
[public] class class_name {  
    ...  
}
```

Ví dụ:

```
class MyDate {  
}
```

Lớp với phương thức main

- Trong Java, mọi thứ phải ở trong một lớp nào đấy.
- Khi thực thi một chương trình, chúng ta thực thi một lớp:
 - Tải lớp và thực thi phương thức main()
 - Lớp phải có phương thức main()!

Định nghĩa gói - package

Cú pháp

```
package vnu.uet.myapplication
```

...

Thường tổ chức theo thư mục chứa các lớp
Định danh địa chỉ của các lớp

Đối tượng

- Đối tượng được thao tác thông qua *tham chiếu*
 - Tham chiếu đóng vai trò gần giống như một con trỏ
- Đối tượng phải được tạo ra một cách tường minh bằng toán tử **new**

```
MyDate d;
```

```
d = new MyDate();
```

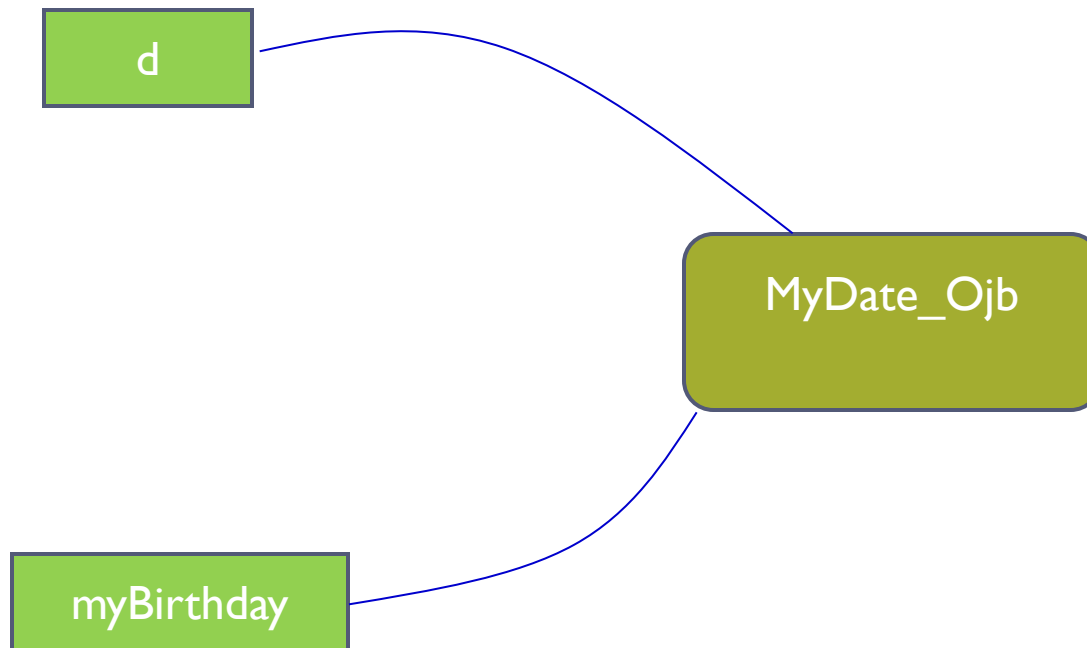
```
MyDate myBirthday = d;
```

Kiểm soát truy cập

```
class MyDate {  
    private int year, mon, day;  
    public int getYear() {  
        return year;  
    }  
    public boolean setYear(int y) {  
        ...  
    }  
    ...  
}
```

Kiểm soát truy cập

	default	private	protected	public
same class	yes	yes	yes	yes
same package subclass	yes	no	yes	yes
same package non-subclass	yes	no	yes	yes
different package subclass	no	no	yes	yes
different package non-subclass	no	no	no	yes



```
class MyDate {  
    private int year, mon, day;  
    public int getYear() {  
        return year;  
    }  
    public void setYear(int y) {  
        year=y;  
    }  
}
```

```
MyDate d = new MyDate();
```

```
...
```

```
d.year = 2005;    // compile error
```

```
d.setYear(2005);
```

```
System.out.println(d.getYear());
```


Phương thức trùng tên (overload)

- Có thể định nghĩa các phương thức trùng tên, tuy nhiên phải phân biệt bởi danh sách tham số

```
class MyDate {  
    ...  
    public boolean setMonth(int m) { ...}  
    public boolean setMonth(String s) { ...}  
}
```

```
d.setMonth(9);
```

```
d.setMonth("September");
```

Phương thức khởi tạo (constructor)

- Dữ liệu nên được khởi tạo trước khi sử dụng
 - lỗi khởi tạo là một trong các lỗi phổ biến
- Phương thức khởi tạo
 - là phương thức đặc biệt được gọi tự động **ngay sau khi đối tượng được tạo ra**
 - nhằm mục đích chính là khởi tạo cho các thuộc tính của đối tượng

Phương thức khởi tạo (constructor)

- Có tên trùng với tên lớp
- Không có giá trị trả về
- Mỗi khi đối tượng được tạo ra bởi toán tử `new`, hệ thống sẽ tự động gọi phương thức khởi tạo.
 - nếu không khai báo, hệ thống sẽ gọi constructor mặc định là một phương thức rỗng

Ví dụ: Constructor rỗng

Không khai báo constructor tường minh

```
class MyDate {  
}
```

...

```
MyDate date = new MyDate();
```

Ví dụ: Constructor không nhận tham số

```
class MyDate {  
    private int day, month, year;  
    public MyDate() {  
        day=1;  
        month=1;  
        year=2014;  
        System.out.println("A new date created");  
    }  
}  
  
...  
MyDate d= new MyDate();
```

Ví dụ: Nhiều constructor

```
class MyDate {
    private int day, month, year;
    public MyDate() {
        day=1;month=1;year=2014;
        System.out.println("A new date created");
    }
    public MyDate(int d,m,y) {
        day=d;
        month=m;
        year=y;
    }
}

...
MyDate d1 = new MyDate();
MyDate d2 = new MyDate(19,9,2014);
```

Ví dụ: Không có constructor mặc định

```
class MyDate {  
    private int day, month, year;  
    public MyDate(int d,m,y) {  
        day=d;  
        month=m;  
        year=y;  
    }  
}  
  
...  
MyDate d1 = new MyDate(); //error  
MyDate d2 = new MyDate(19,9,2014);
```

Copy constructor

- Khởi tạo đối tượng bằng một đối tượng khác

```
public class MyDate {  
    private int year, month, day;  
    public MyDate() {...}  
    public MyDate(MyDate d) {  
        year = d.year;  
        month = d.month;  
        day = d.day;  
    }  
  
    ...  
}
```

```
MyDate d = new MyDate();  
d.setYear(2005);  
d.setMonth(9);  
d.setDay(12);  
MyDate openDay = new MyDate(d);  
MyDate dd = d;  
dd = new MyDate();
```

```
MyDate d = new MyDate();  
d.setYear(2005);  
d.setMonth(9);  
d.setDay(12);
```

```
MyDate dd = d;  
dd = new MyDate();  
dd = new MyDate(d);
```

Khác nhau thế nào?

Kiểm soát truy cập

```
public class MyDate {  
    private int year, month, day;  
    public MyDate() {...}  
    public MyDate(MyDate d) {  
        day = d.day;  
        month = d.month;  
        year = d.year; //thay bằng year=d.getYear()???  
    }  
    ...  
}
```